ESPECIALIZACIÓN EN ANALÍTICA DE BIG DATA
GESTIÓN Y ALMACENAMIENTO DE DATOS

PROYECTO: GESTIÓN INTELIGENTE DE INVENTARIOS FARMACÉUTICOS

STACK TECNOLÓGICO DEL PIPELINE ETL:

El presente documento define el stack tecnológico del pipeline ETL para el prototipo funcional del proyecto de Gestión Inteligente de Inventarios Farmacéuticos. A diferencia de una implementación de producción empresarial, el prototipo tiene como objetivo demostrar que el flujo completo de datos desde las fuentes originales hasta el dashboard de alertas funciona de manera real y verificable, usando herramientas instalables en un computador personal sin costos de licenciamiento.

El stack fue seleccionado equilibrando dos criterios prioritarios para el desarrollo del proyecto. La viabilidad real de implementación, es decir, que todas las herramientas puedan instalarse, configurarse y ejecutarse por una sola persona en un tiempo razonable; y la alineación con la industria, es decir, que las herramientas elegidas sean las mismas que se usan en entornos profesionales reales, garantizando que el prototipo sea una demostración auténtica y no una simulación académica desconectada de la práctica.

El pipeline procesa datos sintéticos realistas que simulan las fuentes de una IPS colombiana mediana como sistema de farmacia, ERP de compras y Excel de consumos históricos. Si durante el desarrollo se obtiene acceso a datos reales, el pipeline es directamente reutilizable sin cambios estructurales.

1. Stack Ideal vs. Stack del Prototipo:

La siguiente tabla compara el stack tecnológico completo documentado en el estado del arte con el stack simplificado seleccionado para el prototipo funcional. Ambos resuelven el mismo problema con la misma lógica ETL; la diferencia está en la escala y la infraestructura requerida.

Capa ETL	Stack ideal	Stack del prototipo	Por qué es equivalente
Extracción (E)	Airbyte Open Source (interfaz web, 600+ conectores)	Scripts Python (pandas, sqlalchemy, openpyxl)	Ambos extraen datos de las mismas 3 fuentes. Python es más directo para un prototipo con datos sintéticos.
Orquestación	Apache Airflow (DAGs, monitoreo web, reintentos)	Airflow local o ejecución manual del script	Airflow puede instalarse localmente con pip. Para la demo, ejecutar el script manualmente es suficiente para mostrar el flujo.
Calidad de datos (T)	Great Expectations (suite de validaciones)	Validaciones Python (assert, pandas checks)	Las mismas reglas de negocio (stock >= 0, sin duplicados, fechas válidas) se implementan con código Python simple.
Transformación (T)	dbt Core (modelos SQL versionados)	dbt Core local (idéntico, instalable con pip)	dbt Core es gratuito y corre localmente. Es exactamente la misma herramienta de producción. No hay simplificación aquí.
Almacenamiento (L)	PostgreSQL (servidor dedicado)	PostgreSQL local (instalación en el PC)	PostgreSQL local es idéntico en funcionalidad. El prototipo usa una base de datos en el computador personal.
Visualización	Microsoft Power BI Pro ($10/usuario/mes)	Power BI Desktop (gratuito)	Power BI Desktop tiene las mismas capacidades de visualización y alertas. Solo difiere en publicación en la nube.

Tabla 1. Comparación entre el stack de producción y el stack del prototipo funcional

2. Criterios de Seleccion del Stack:

La selección del stack tecnológico para el prototipo funcional no se basó únicamente en la popularidad de las herramientas ni en su capacidad técnica máxima, sino en su viabilidad real para un proyecto individual de alcance académico.

Se definieron cuatro criterios de evaluación aplicados de forma sistemática a cada herramienta candidata en cada capa del pipeline ETL, con el objetivo de garantizar que el prototipo pueda ser construido, ejecutado y demostrado por una sola persona en un entorno de PC personal, sin infraestructura cloud ni costos de licenciamiento.

Criterio	Descripción	Peso para el prototipo
Costo cero	Todas las herramientas deben ser gratuitas o tener versión gratuita funcional	CRÍTICO: Proyecto académico sin presupuesto
Instalable en PC personal	Debe correr en Windows/Mac/Linux sin servidores externos ni cloud	CRÍTICO: Unico entorno disponible
Curva de aprendizaje corta	Debe poder dominarse en días, no en semanas, para un desarrollador individual	ALTO: Tiempo limitado de entrega
Alineación con la industria	Debe ser la misma herramienta o equivalente directo a lo que se usa en producción	ALTO: El prototipo debe ser académicamente válido y profesionalmente relevante

Tabla 2. Criterios de Seleccion del Stack

Los dos criterios marcados como críticos costo cero e instalable en PC personal actuaron como filtros eliminatorios. Cualquier herramienta que requiriera pago de licencia o infraestructura externa fue descartada independientemente de su puntaje en los otros criterios. Esto explica, por ejemplo, el descarte de Google BigQuery a pesar de su alta alineación con la industria: su modelo de pago por consulta lo hace inviable para un prototipo académico sin presupuesto.

El criterio de alineación con la industria fue determinante para evitar caer en soluciones puramente académicas desconectadas de la práctica profesional. La decisión más importante en este sentido fue mantener dbt Core sin ninguna simplificación: es exactamente la misma herramienta que usan equipos de ingeniería de datos en empresas reales, se instala con un solo comando pip y corre localmente contra PostgreSQL. Reemplazarla por scripts SQL manuales habría sido más sencillo, pero habría sacrificado la validez profesional del prototipo.

El criterio de curva de aprendizaje corta justifica la sustitución de Airbyte por scripts Python en la capa de extracción. Airbyte es la herramienta ideal en producción, pero requiere Docker instalado y configurado, lo que representa una barrera de infraestructura desproporcionada para el alcance del prototipo. Python con pandas logra exactamente el mismo resultado al leer Excel, CSVs y conectar a PostgreSQL en menos tiempo de configuración y con mayor control sobre el proceso de limpieza.

En conjunto, los cuatro criterios garantizan que el stack seleccionado sea al mismo tiempo implementable de forma realista y suficientemente robusto para demostrar que el pipeline ETL propuesto no es un ejercicio teórico, sino una solución funcional y reproducible.

3. Tabla Comparativa de Herramientas por Capa ETL:

A continuación se evaluan las herramientas candidatas para cada capa del prototipo, aplicando los cuatro criterios definidos.

3.1 Capa de Extracción (E):

La capa de extracción es responsable de leer los datos desde las tres fuentes originales el archivo Excel de consumos históricos, el CSV de dispensaciones del sistema de farmacia y el CSV de compras del ERP y cargarlos en el área de staging de PostgreSQL sin modificarlos. Las herramientas evaluadas para esta capa fueron Python con pandas, Airbyte Open Source, Apache NiFi y Talend Open Studio.

Python con pandas obtuvo el puntaje más alto con 11 sobre 12, siendo la única herramienta que cumple los cuatro criterios de forma simultánea para el alcance del prototipo. Es completamente gratuita, se instala con un solo comando pip, su curva de aprendizaje es baja para cualquier persona con conocimientos básicos de programación, y es ampliamente usada en entornos profesionales de ingeniería de datos para tareas de extracción ligera.

Airbyte, aunque es la opción ideal en producción por sus más de 600 conectores predefinidos, fue catalogada como herramienta de producción y no de prototipo porque requiere Docker instalado y en funcionamiento, lo que representa una barrera de infraestructura desproporcionada para el alcance del proyecto. Apache NiFi y Talend fueron descartados por combinar alta complejidad de instalación con curvas de aprendizaje pronunciadas que no se justifican para un prototipo individual.

Herramienta	Costo cero	Instalable en PC	Curva aprendizaje	Alineación industria	Total	Decisión
Python (pandas, openpyxl, sqlalchemy)	Gratuito	Sí	Baja	Media	11	SELECCIONADO
Airbyte Open Source	Gratuito	Requiere Docker	Media	Muy alta	9	Descartado
Apache NiFi	Gratuito	Requiere Java+RAM	Alta	Alta	8	Descartado
Talend Open Studio	Freemium	Pesado	Media	Media	8	Descartado

Tabla 3. Herramientas candidatas para la Capa de Extracción (E)

3.2 Capa de Transformación (T):

La capa de transformación es el núcleo del pipeline: aquí se limpian, estandarizan y calculan los indicadores clave del inventario stock disponible, punto de reorden y flags de alerta. Las herramientas evaluadas fueron dbt Core, validaciones Python con pandas, Great Expectations y Apache Spark.

Esta capa tiene la particularidad de contar con dos herramientas seleccionadas de forma complementaria. dbt Core obtuvo 11 sobre 12 y fue seleccionado como la herramienta principal de transformación sin ninguna simplificación respecto al stack de producción: es gratuito, se instala con pip install dbt-postgres, corre directamente contra PostgreSQL local y permite definir los modelos de cálculo de stock en SQL versionado y documentado automáticamente.

Las validaciones Python con pandas también obtuvieron 11 sobre 12 y fueron seleccionadas como capa de calidad de datos, ejecutándose antes de que dbt procese cualquier transformación para garantizar que los datos del staging no contengan registros corruptos.

Great Expectations, aunque es la herramienta profesional equivalente para calidad de datos, quedó como opción opcional por su configuración inicial compleja, que puede implementarse si el tiempo de desarrollo lo permite. Apache Spark fue descartado por ser una herramienta diseñada para procesamiento distribuido de grandes volúmenes, completamente sobredimensionada para el tamaño de datos de una IPS mediana en un prototipo.

Herramienta	Costo cero	Instalable en PC	Curva aprendizaje	Alineación industria	Total	Decisión
dbt Core (pip install dbt-postgres)	Gratuito	Sí, con pip	Media	Muy alta	11	SELECCIONADO
Validaciones Python (pandas + assert)	Gratuito	Sí	3 - Baja	Media	11	SELECCIONADO
Great Expectations	Gratuito	Sí, con pip	Alta	Alta	9	Opcional si hay tiempo
Apache Spark	Gratuito	Complejo	Muy alta	Alta	8	Descartado

Tabla 4. Herramientas candidatas para la Capa de Transformación (T)

3.3 Capa de Almacenamiento y Visualización (L):

La capa de carga y visualización comprende dos funciones distintas: el almacenamiento de los datos transformados en un repositorio centralizado y la presentación de los indicadores al usuario final. Las herramientas evaluadas fueron PostgreSQL local, Power BI Desktop, Metabase y Google BigQuery.

PostgreSQL local y Power BI Desktop fueron seleccionados con puntajes de 11 y 12 sobre 12 respectivamente, siendo las únicas herramientas que cumplen todos los criterios sin concesiones. PostgreSQL es el motor de base de datos relacional open source más robusto disponible, idéntico en funcionalidad a un servidor PostgreSQL en producción, y tiene conectores nativos con todas las demás herramientas del stack dbt, Python y Power BI.

Power BI Desktop obtuvo el puntaje máximo de 12 porque es gratuito, se instala directamente desde la Microsoft Store, tiene una curva de aprendizaje baja gracias a su interfaz visual de arrastrar y soltar, y es exactamente la misma herramienta que se usa en entornos empresariales reales. Su única diferencia respecto a Power BI Pro es que no permite publicar el dashboard en la nube, lo cual no es una limitación para la demo del prototipo.

Google BigQuery fue descartado en la primera evaluación por su modelo de pago por consulta, que lo hace inviable para un proyecto sin presupuesto, independientemente de sus otras ventajas técnicas.

Herramienta	Costo cero	Instalable en PC	Curva aprendizaje	Alineación industria	Total	Decisión
PostgreSQL local (data warehouse)	Gratuito	Sí	Media	Muy alta	11	SELECCIONADO
Power BI Desktop (visualización)	Gratuito	Solo Windows	Baja	Muy alta	12	SELECCIONADO
Metabase (alternativa BI)	Gratuito	Requiere Java	Muy baja	Media	10	Alternativa si no hay Windows
Google BigQuery	Pago por uso	Cloud	Media	Alta	9	Descartado

Tabla 5. Herramientas candidatas para la Capa de Almacenamiento y Visualización (L)

4. Stack Definitivo del Prototipo y Justificación:

Con base en la evaluación comparativa, el stack definitivo del prototipo funcional está compuesto por cinco herramientas que cubren todas las etapas del pipeline ETL, son completamente gratuitas, instalables en un PC personal y alineadas con la práctica profesional de ingeniería de datos:

Capa	Herramienta	Función en el prototipo	Instalación	Costo
EXTRACCIÓN (E)	Python (pandas, openpyxl, sqlalchemy)	Lee el archivo Excel de consumos, conecta al CSV de dispensaciones y al CSV de compras del ERP; carga los datos crudos al staging en PostgreSQL	pip install pandas openpyxl sqlalchemy psycopg2	Gratuito
CALIDAD (T)	Python (validaciones con pandas)	Valida que stock >= 0, códigos de medicamento válidos, fechas no nulas, sin duplicados por lote antes de transformar	Incluido en pandas	Gratuito
TRANSFORMACIÓN (T)	dbt Core (dbt-postgres)	Calcula stock_disponible, consumo_diario_promedio, punto_de_reorden y flag_alerta para cada medicamento crítico	pip install dbt-postgres	Gratuito
ALMACENAMIENTO (L)	PostgreSQL local	Repositorio central con tablas: staging_dispensaciones, staging_compras, staging_consumos, mart_stock_critico, mart_alertas	pip install	Gratuito
VISUALIZACIÓN	Power BI Desktop	Dashboard: semáforo rojo/amarillo/verde por medicamento, días de cobertura, top 10 en riesgo, tendencia de consumo	Microsoft Store / powerbi.com	Gratuito

Tabla 6. Stack tecnológico definitivo del prototipo: herramientas, funciones e instalación.

Justificación por herramienta:

a) Python - Extracción: Se selecciona Python sobre Airbyte para la capa de extracción del prototipo por una razón práctica determinante: Airbyte requiere Docker instalado y configurado, lo que agrega una capa de infraestructura innecesaria para un prototipo individual. Python, en cambio, se instala con un solo comando pip y permite leer archivos Excel con openpyxl, conectar a PostgreSQL con sqlalchemy y procesar CSVs con pandas en pocas líneas de código.

b) Validaciones Python - Calidad de datos: En lugar de Great Expectations, las reglas de calidad se implementan directamente en Python usando aserciones pandas. Esto cubre los mismos controles críticos: stock nunca negativo, códigos de medicamento válidos, fechas no nulas y registros sin duplicados por lote. Si una validación falla, el script lanza una excepción con mensaje descriptivo y detiene la ejecución.

c) dbt Core - Transformación: Esta es la única herramienta del stack de producción que se mantiene idéntica en el prototipo, sin ninguna simplificación. dbt Core es gratuito, se instala con pip install dbt-postgres y corre localmente contra PostgreSQL. Los modelos SQL del proyecto calcularán:

- Estandarización de nombres por código ATC/CUPS,
- Stock_disponible = SUM(ingresos_ERP) - SUM(dispensaciones_farmacia) por medicamento
- Consumo_diario_promedio de los últimos 30 días
- Punto_de_reorden = consumo_diario_promedio x lead_time_proveedor
- Flag_alerta = TRUE cuándo stock_disponible <= punto_de_reorden.

Usar dbt real, no un script Python equivalente, es una decisión deliberada para demostrar que el prototipo usa herramientas de nivel profesional.

d) PostgreSQL local - Almacenamiento: PostgreSQL es el data warehouse del prototipo. Se instala gratuitamente en Windows, Mac o Linux desde postgresql.org y es funcionalmente idéntico a un servidor PostgreSQL en producción. El modelo de datos contendrá:

- Tablas de staging (raw_dispensaciones, raw_compras, raw_consumos_excel)
- tablas dimensionales (dim_medicamentos)
- Tablas de hechos (fact_inventario_diario)
- Data marts (mart_stock_critico, mart_alertas_desabastecimiento) generados por dbt.

e) Power BI Desktop - Visualización: Power BI Desktop es la versión gratuita de Power BI, disponible para Windows, con exactamente las mismas capacidades de construcción de dashboards que la versión Pro. La única diferencia es que no permite publicar el dashboard en la nube para compartirlo; para la demo del prototipo, presentar el dashboard en pantalla desde Power BI Desktop es completamente suficiente.

El dashboard mostrara: semaforo de stock critico (rojo: desabastecimiento inminente; amarillo: stock bajo; verde: stock seguro), dias de cobertura restantes por medicamento, top 10 medicamentos en riesgo y tendencia de consumo por servicio clinico.

5. Diagrama de Arquitectura del Prototipo:

El diagrama de arquitectura representa el flujo completo de datos del prototipo, desde las tres fuentes originales hasta el dashboard de alertas en Power BI Desktop. La arquitectura está organizada en capas secuenciales donde la salida de cada etapa es la entrada de la siguiente, garantizando que ningún dato llegue a la visualización sin haber pasado por extracción, validación y transformación.

El flujo inicia en tres archivos sintéticos que simulan fielmente las fuentes reales de una IPS colombiana: dispensaciones.csv representa el sistema de farmacia, compras.csv representa el ERP de compras, y consumos_historicos.xlsx representa el registro manual en Excel. Estos tres archivos son leídos simultáneamente por el script Python de extracción, que los valida y los carga en las tablas raw del área de staging en PostgreSQL. Sobre ese staging, dbt Core ejecuta los modelos SQL de transformación que calculan el stock disponible, el punto de reorden y los flags de alerta por medicamento. Los resultados se almacenan en los data marts de PostgreSQL, desde dónde Power BI Desktop los consume para construir el dashboard de semáforo en tiempo real.

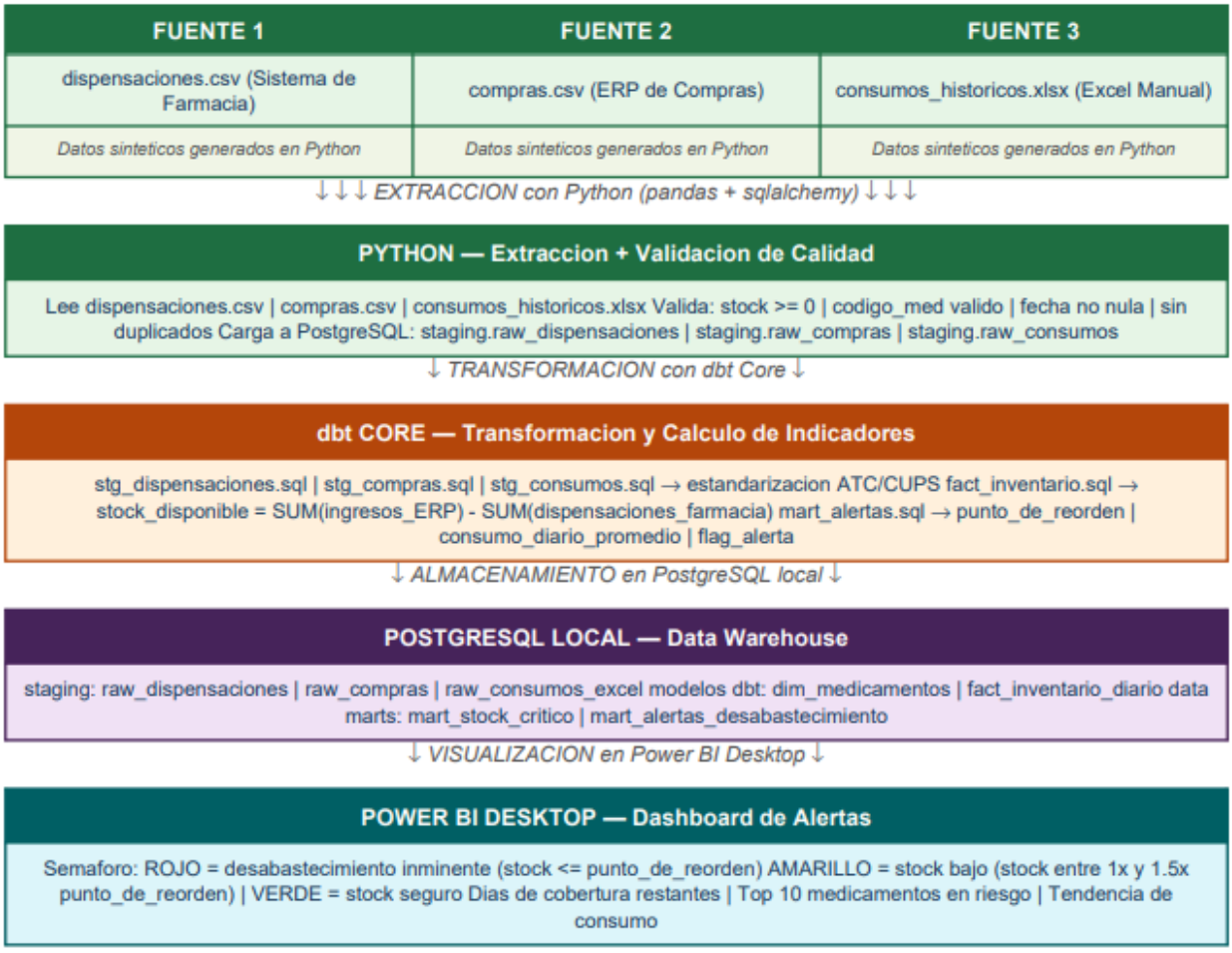

Figura 1. Arquitectura del prototipo ETL funcional ejecutable

La arquitectura del prototipo replica fielmente la estructura de un pipeline ETL de producción en cinco capas bien diferenciadas: fuentes, extracción y calidad, transformación, almacenamiento y visualización. Esta separación no es únicamente conceptual sino funcional: cada capa puede ejecutarse, verificarse y depurarse de forma independiente, lo que facilita tanto el desarrollo incremental del prototipo como su demostración al profesor en cualquier etapa del proceso.

6. Plan de Implementación del Prototipo:

El plan de implementación organiza el desarrollo del prototipo en cinco fases secuenciales, cada una con una actividad concreta, una herramienta principal y un entregable verificable que puede demostrarse de forma independiente. Las fases están diseñadas para que el avance sea incremental y acumulativo: completar cada fase deja el prototipo en un estado funcional parcial que ya puede ejecutarse y presentarse, sin necesidad de esperar a que todas las fases estén terminadas.

La implementación del prototipo se organiza en cinco fases secuenciales. Cada fase produce un entregable verificable que puede demostrarse de forma independiente:

Fase	Actividad	Herramienta	Entregable verificable
1	Generar datos sintéticos realistas de las 3 fuentes (medicamentos colombianos, códigos ATC, cantidades coherentes)	Python (faker, pandas)	3 archivos: dispensaciones.csv, compras.csv, consumos_historicos.xlsx
2	Instalar y configurar PostgreSQL local; crear esquemas staging y marts; conectar Python	PostgreSQL / psycopg2	Base de datos funcional con esquemas creados y conexión verificada
3	Desarrollar script Python de extracción y validación de calidad; cargar al staging de PostgreSQL	Python / pandas, sqlalchemy	Script ETL ejecutable; tablas raw pobladas en PostgreSQL
4	Instalar dbt Core; crear modelos SQL de estandarización, cálculo de stock, punto de reorden y flags de alerta	dbt Core (dbt-postgres)	dbt run exitoso; tablas mart_stock_critico y mart_alertas pobladas
5	Conectar Power BI Desktop a PostgreSQL; construir dashboard de semáforo, KPIs y tendencias	Power BI Desktop	Dashboard funcional con semáforo rojo/amarillo/verde y top 10 en riesgo

Tabla 7. Implementación del prototipo por fases

La Fase 1 genera los tres archivos de datos sintéticos que alimentan todo el pipeline. La Fase 2 instala y configura PostgreSQL local como repositorio central. La Fase 3 desarrolla el script Python que extrae, válida y carga los datos al staging. La Fase 4 instala dbt Core y construye los modelos SQL que calculan los indicadores de inventario. La Fase 5 conecta Power BI Desktop a PostgreSQL y construye el dashboard de alertas de desabastecimiento.

CONCLUSIONES:

El stack tecnológico definido para el prototipo Python + dbt Core + PostgreSQL local + Power BI Desktop es completamente viable para el desarrollo del proyecto: todas las herramientas son gratuitas, instalables en un PC personal en menos de una hora, y alineadas con la práctica profesional real de ingeniería de datos.

La decisión clave del stack es mantener dbt Core sin simplificación, usando la misma herramienta de producción para la capa de transformación. Esto garantiza que el prototipo no es una simulación sino una demostración auténtica del flujo ETL completo, con modelos SQL versionados, linaje de datos documentado automáticamente y cálculos de negocio verificables: stock_disponible, punto_de_reorden y flag_alerta por medicamento.

El prototipo cumple directamente con el objetivo del proyecto: demostrar que integrar automáticamente las fuentes de datos de una IPS (farmacia, ERP, Excel) mediante un pipeline ETL permite calcular el stock real de medicamentos críticos y generar alertas tempranas de desabastecimiento, resolviendo el problema de datos fragmentados identificado como causa raíz.

REFERENCIAS:

- Apache Airflow. (2024). Apache Airflow Documentation 2.x — ETL/ELT use cases. Apache Software Foundation. https://airflow.apache.org/use-cases/etl_analytics/

- Airbyte Inc. (2025). Airbyte Open Source: Connector catalog. Airbyte. https://docs.airbyte.com/integrations/

- PostgreSQL Global Development Group. (2025). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/

- Python Software Foundation. (2025). pandas documentation: IO tools (text, CSV, HDF5, …). https://pandas.pydata.org/docs/user_guide/io.html

- SQLAlchemy. (2025). SQLAlchemy 2.0 Documentation. https://docs.sqlalchemy.org/en/20/

- Mendoza-Erazo, M. B., Meléndez-Romo, É. E., Fuentes-Jácome, M. M., Barahona-Rivera, R. J., & Solis-Santana, G. A. (2024). Modelo estructural de inteligencia de negocios para inventarios en farmacéutica. Multidisciplinary Latin American Journal (MLAJ), 2(1), 1–12. https://doi.org/10.62131/MLAJ-V2-N1-001

- LACCEI. (2025). Optimización de inventarios farmacéuticos mediante técnicas predictivas de Machine Learning y Business Intelligence. 5th LACCEI International Multiconference — LEIRD 2025. https://dx.doi.org/10.18687/LEIRD2025.1.1.438

- MAS Analytics. (2025). Modelos BI de planificación e inventario para la industria farmacéutica. https://www.masanalytics.com/articulo/modelos-bi-de-planificacion-e-inventario-para-la-industria-farmaceutica/